ABOUT

Title: The Last Save
Created by: Genon, Philippe Jonathan
Tatoy, Alexander
Guimba, Vaness
Calala, ainoden
Genre: School Manga / Tech Drama / Mystery
Target Audience: IT Students and Educators

SYNOPSIS

Delfin, an IT student, is confident that his Python project is ready for submission. But one last-minute change triggers a system error, putting their hard work at risk. With the deadline approaching, Delfin and his teammates, Mia and Mark, must find the cause, recover their files, and save their project. Through unexpected problems and difficult lessons, they discover the importance of testing, teamwork, and regular backups.

CAST OF CHARACTERS

Delfin: A talented but overconfident programmer who learns to test his code and save his work properly.

Mia: A careful and observant teammate who spots errors and reminds the group to make backups.

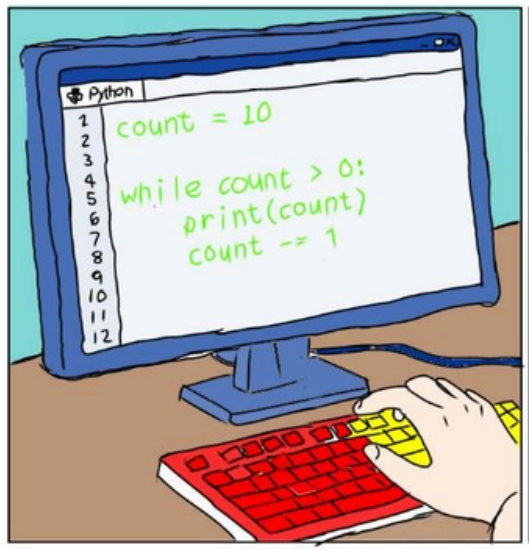
Mark: A friendly and practical IT student who helps troubleshoot problems and supports the team.

LESSON

“Save your work. Test your code. Trust your team.”

Finally, the coding is done.
Great job, Delfin!

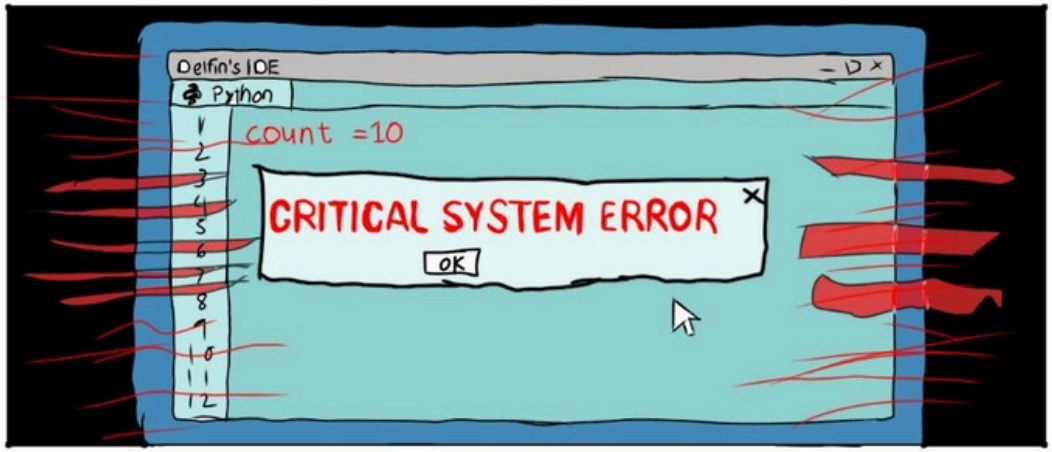
I just need to add this one last small optimization.

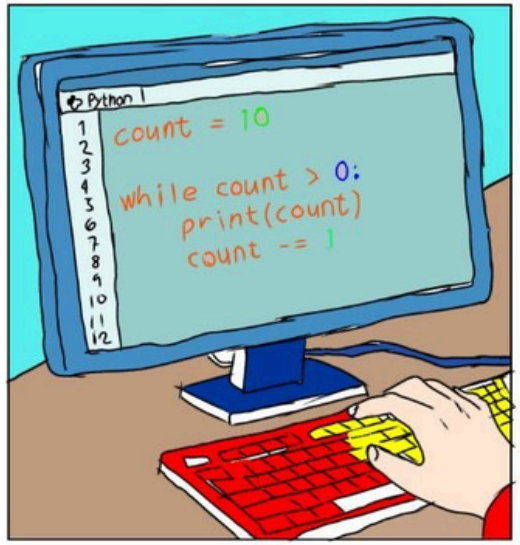

Python
count = 10
while count > 0:
print(count)
count -= 1

Perfect. It's stable and ready.
Mia, we should submit this!
Are you sure it's fully checked?

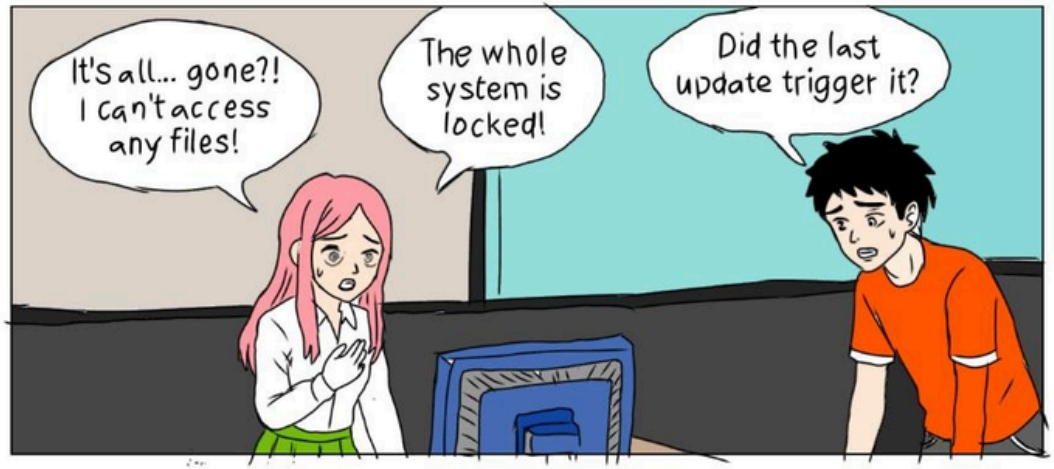
You rushed that last part, Delfin.
It should be fine, though

Trust me! Running the project one more time.. then submit!
count = 10
while count > 0:
print(count)
count -= 1
RUN

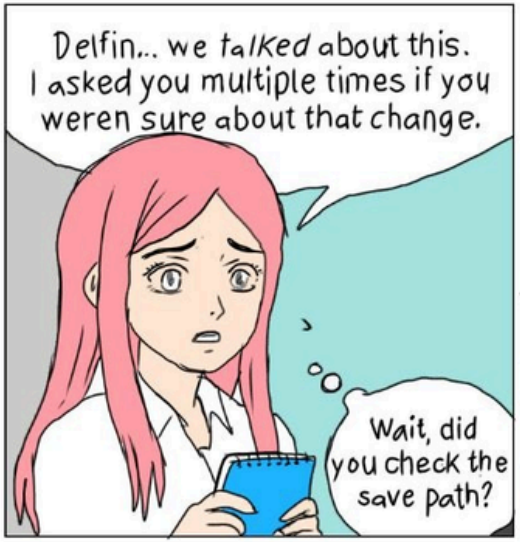

Delfin's IDE
Python
count =10
CRITICAL SYSTEM ERROR
OK


Python
count = 10
while count > 0:
print(count)
count -= 1

What... just happened?
Is that...?
I didn't see that coming!

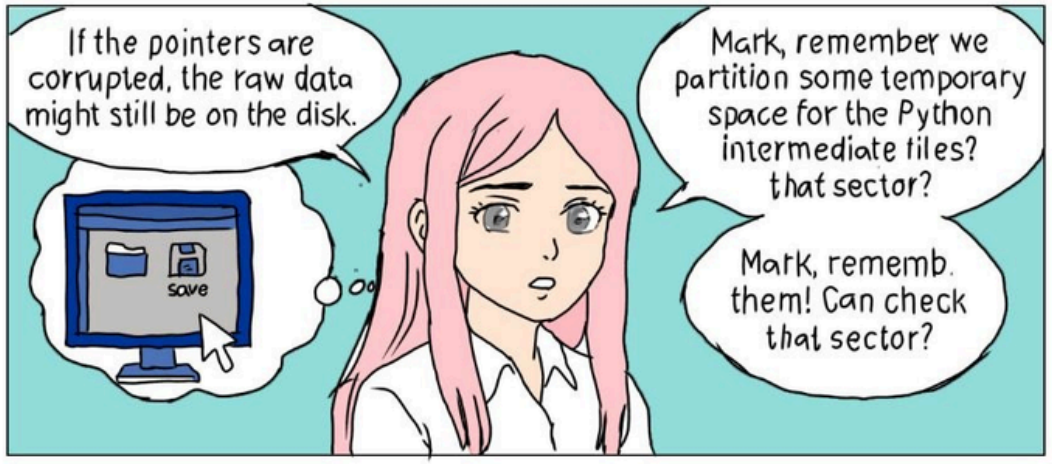

It's all... gone?! I can't access any files!
The whole system is locked!
Did the last update trigger it?

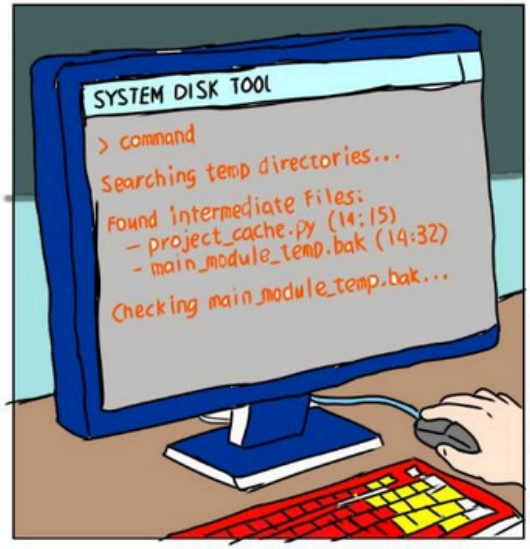
Delfin's IDE
Python
count = 10
while
FILESYSTEMERROR
OK
Come on! Work! Where is the previous version?!
Mia, Mark, I need the backups! Please tell me you made backups!


Delfin... we talked about this. I asked you multiple times if you weren sure about that change.
Wait, did you check the save path?

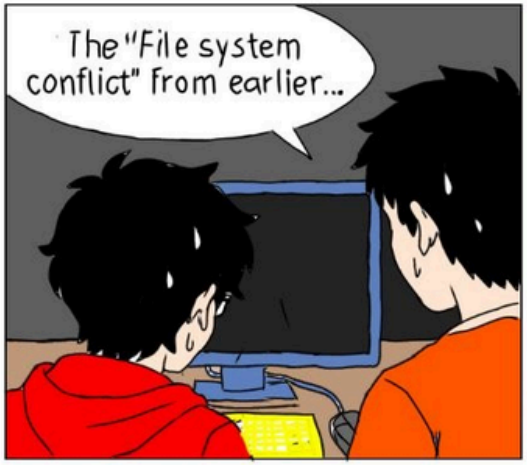
I'm checking my external... but that file system conflict might have locked our access to the shared folder, too!
Access Denied
OK

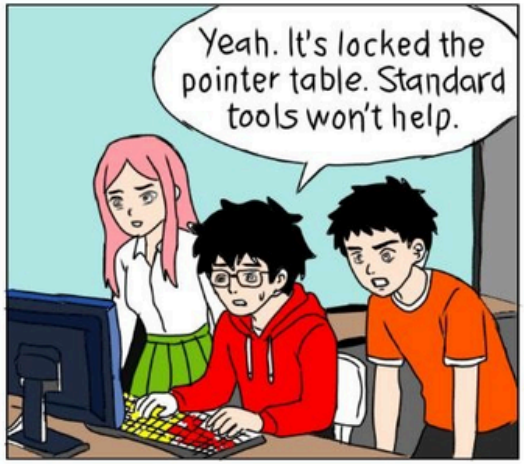
I thought my single save point was enough... I made that last change so fast.
It seems to have destabilized entire file system pointer.


If the pointers are corrupted, the raw data might still be on the disk.
save
Mark, remember we partition some temporary space for the Python intermediate files? that sector?
Mark, rememb. them! Can check that sector?

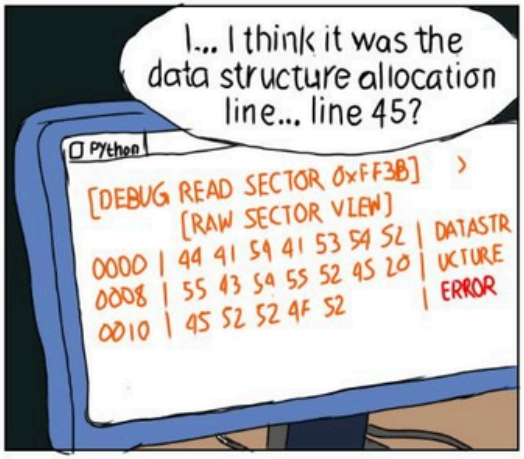

SYSTEM DISK TOOL
> command
Searching temp directories...
Found intermediate files:
- project_cache.py (14:15)
- main_module_temp.bak (14:32)
Checking main_module_temp.bak...

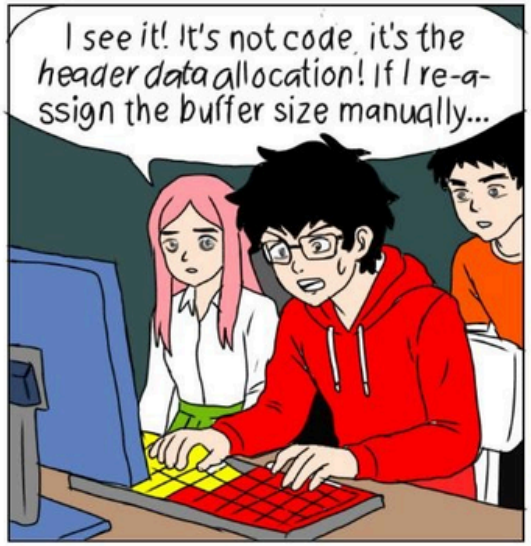
I'm rerouting the sector access! This might work... standard Python intermediate files often save the last good compile!
Really? Even after a hard corrupt?!


The "File system conflict" from earlier...


Yeah. It's locked the pointer table. Standard tools won't help.

Delfin, your change... what specific line did you edit? If we can isolate that, we can force a raw read.


I... I think it was the data structure allocation line... line 45?
Python
[DEBUG READ SECTOR 0xFF3B]
[RAW SECTOR VIEW]
0000 | 44 41 54 41 53 54 52 | DATASTR
0008 | 55 43 54 55 52 45 20 | UCTURE
0010 | 45 52 52 4F 52 | ERROR


I see it! It's not code, it's the header data allocation! If I re-a-ssign the buffer size manually...

Will it work without destabilizing the whole temporary partition?
It has to. This is the only way.
Focus, Delfin. We don't get another shot at this temporary backup.

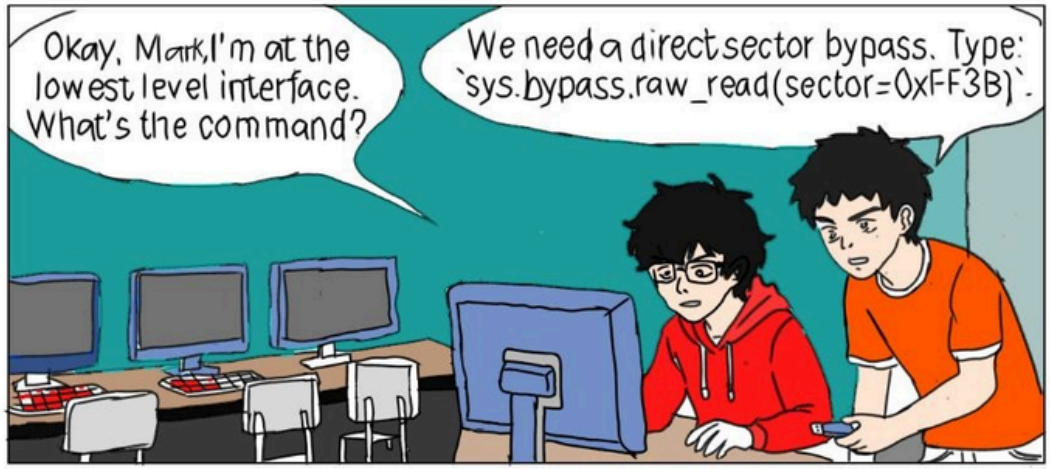
Still running. This is taking too long!
The progress bar hasn't moved for five minutes.
Is it just stalled, or...?

Come on... don't crash on me. If we lose this temporary data...

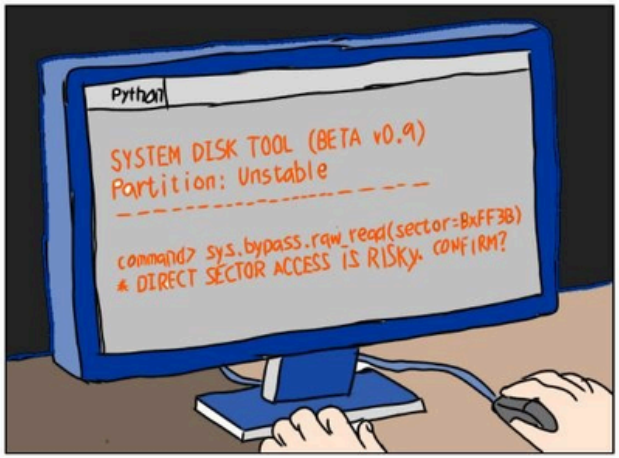
FORCED SECTOR REPAIR INITIATED...
50%
* STATUS CHANGE: PARTITION UNSTABLE

Delfin! Mark! We can't wait. We have to bypass the system check and copy the raw hex dump now!
You mean... standard read/ bypass? risky!
Wait, the system integrity might fail if we interrupt.

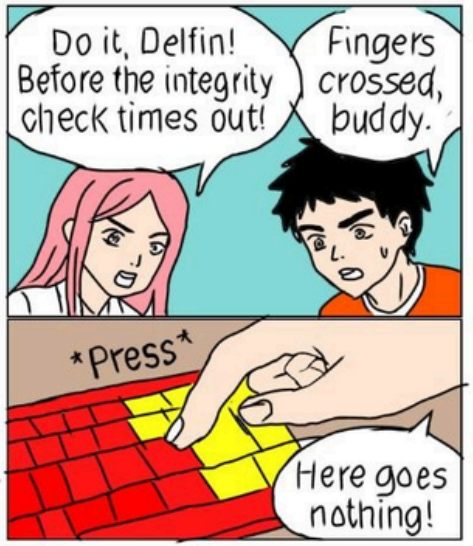
It's already failing! Mark, create a temporary mount point. Delfin, prepare the transfer script!
I... I've done this manually.
Start typing, Delfin! I'm plugging in!


Okay, Mark, I'm at the lowest level interface. What's the command?
We need a direct sector bypass. Type: `sys.bypass.raw_read(sector=0xFF3B)`.

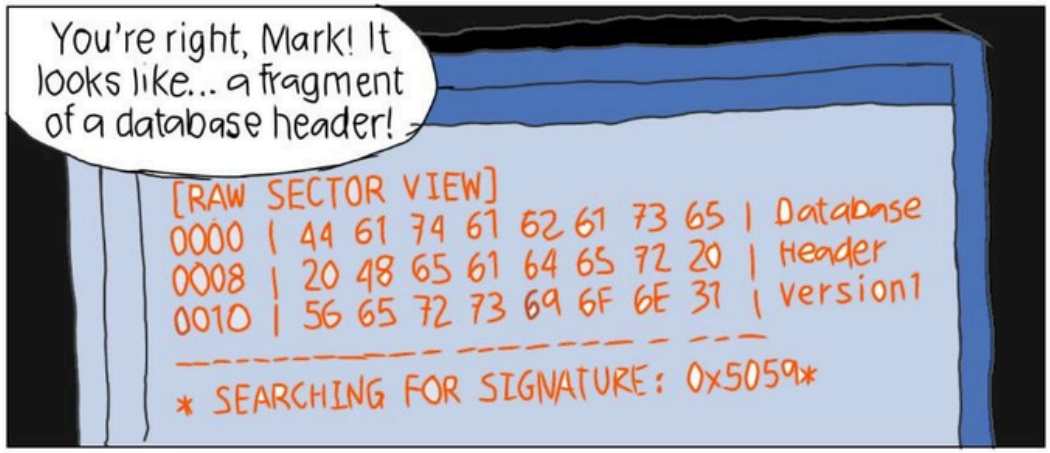
a direct sector bypass. Type: `sys.bypass raw_read

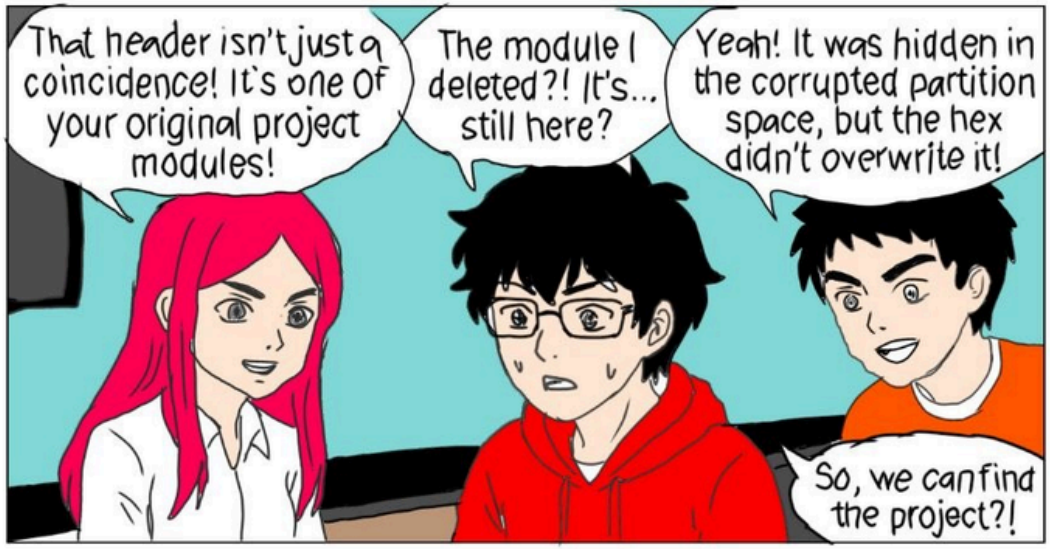

Python
SYSTEM DISK TOOL (BETA v0.9)
Partition: Unstable
command> sys.bypass.raw_read(sector=0xFF3B)
* DIRECT SECTOR ACCESS IS RISKY. CONFIRM?

This has to work... or the whole drive might lock up forever.


Do it, Delfin! Before the integrity check times out!
Fingers crossed, buddy.
Press
Here goes nothing!

Okay, I executed the bypass. It's... a mess of hex data.
Look! That pattern of `44 61`... isn't that ASCII for 'Da'?


You're right, Mark! It looks like... a fragment of a database header!
[RAW SECTOR VIEW]
0000 | 44 61 74 61 62 61 73 65 | Database
0008 | 20 48 65 61 64 65 72 20 | Header
0010 | 56 65 72 73 69 6F 6E 31 | Version1
* SEARCHING FOR SIGNATURE: 0x5059*


That header isn't just a coincidence! It's one of your original project modules!
The module I deleted?! It's... still here?
Yeah! It was hidden in the corrupted partition space, but the hex didn't overwrite it!
So, we can find the project?!

The hex dump gave us the location! I'm trying a `sys.recover()` command on the module address.
14:35
Make sure you point to the raw partition header we found, not the virtual one.

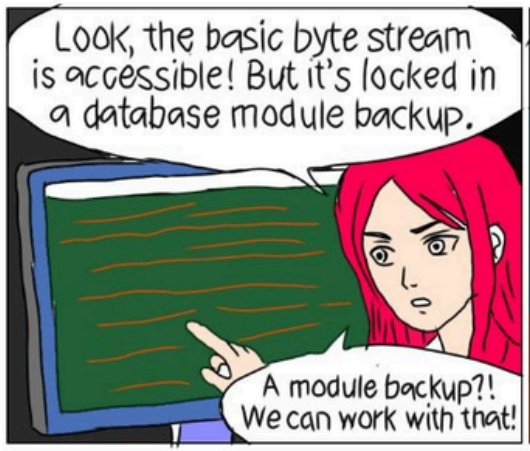
If the hex signature '0x5059' is intact, we can force a raw byte stream copy of the following 1024 sectors.

Force a copy? Is that safe? We could scramble something else!
SYSTEM RECOVERY
command> sys.recover(addr=0xFF3B, signature='0x5059', sectors=1024, output='recovered_module.bak')
[...WAITING...]
Warning: Critical file system points are unstable. Forcing raw byte copy

Okay, everything is set. But what if we find the data but it's corrupted?
We have no other choice. Press it.
ENTER

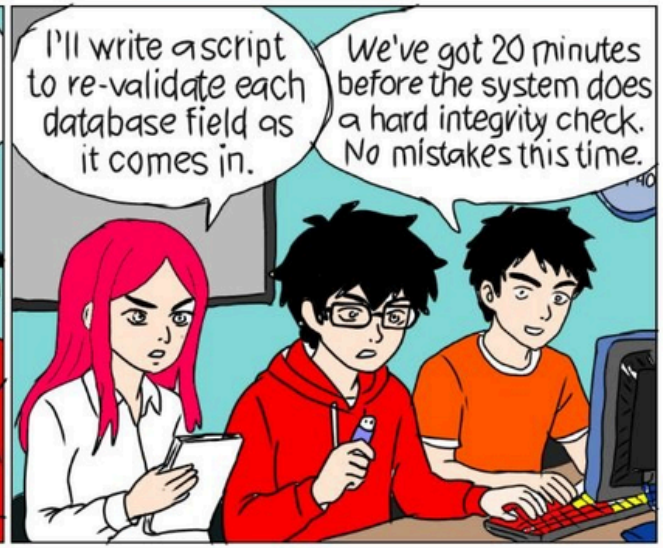
The recovery process is running... but the remaining time is huge. What if it fails?
The hex signature was intact, that's the best sign we could have!
14:35
Stay focused. We need to plan how to verify the data if it survives.

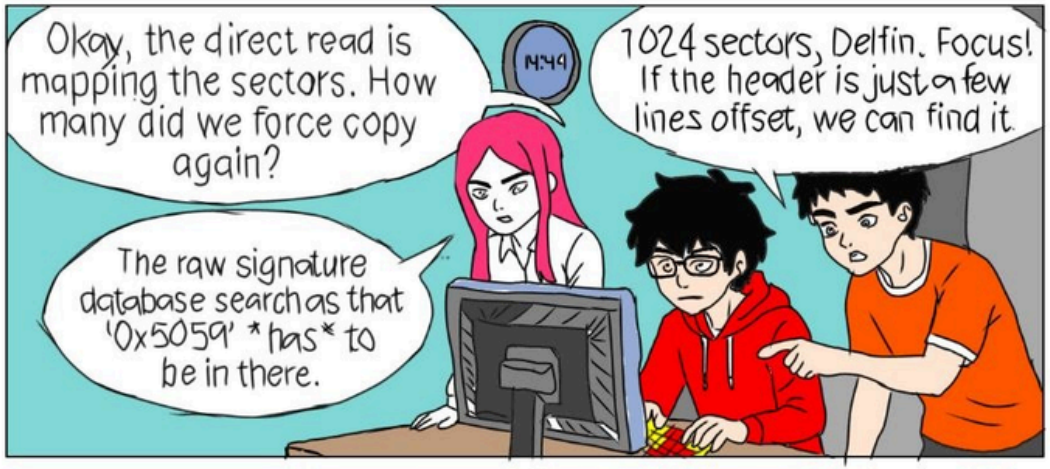
The recovery script.. it Finished! it did not fail!
Is the file okay? Can we check the database header again?
One step at a time, Mark. First, verify the file structure.


Look, the basic byte stream is accessible! But it's locked in a database module backup.
A module backup?! We can work with that!

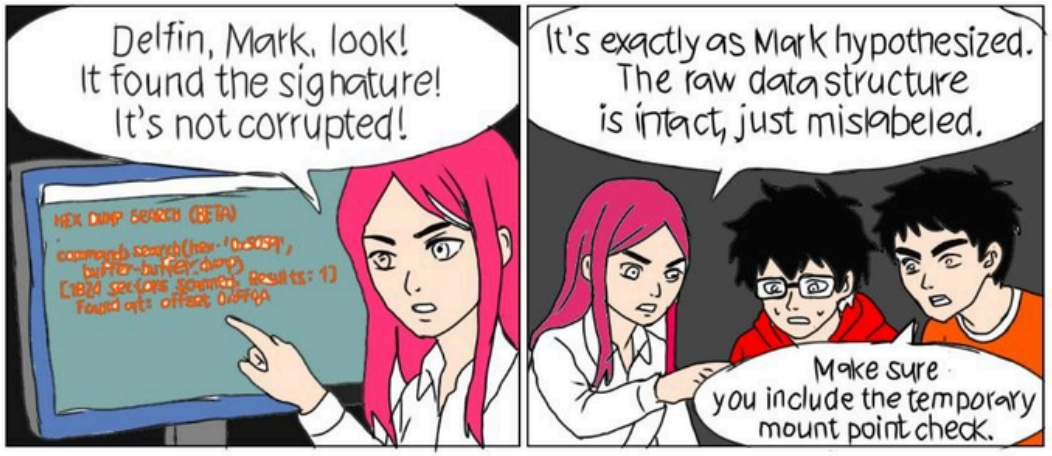
Wait, I remember a trick! We don't need the original pointers if we force a direct buffer read!
Exactly! Mia can guide the hex bypass, and you can write the Python verify script!

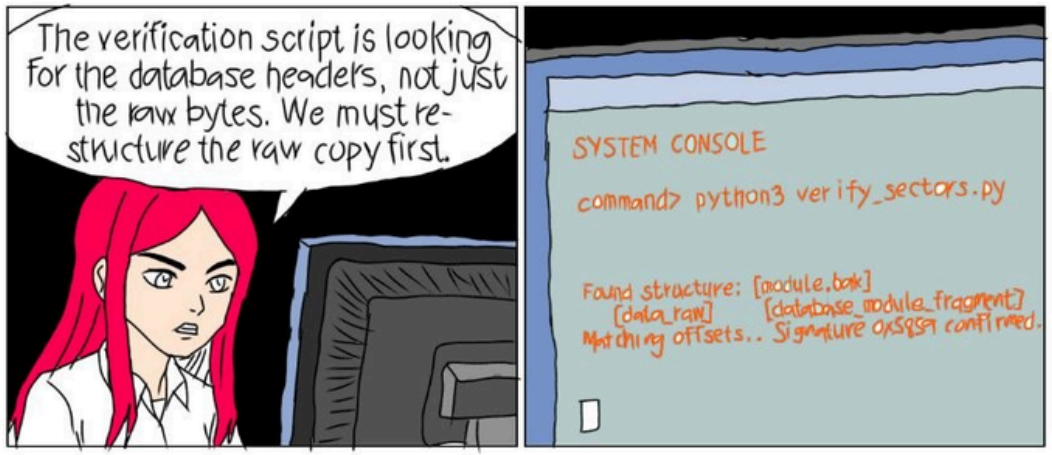
Let's do this. I'll get the raw buffer addresses from the new hex map.

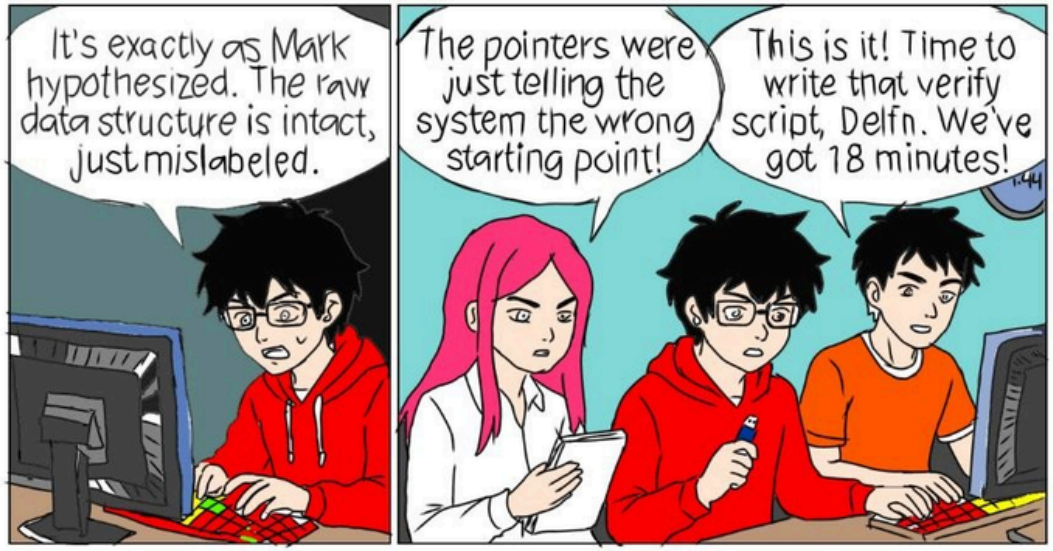

I'll write a script to re-validate each database field as it comes in.
We've got 20 minutes before the system does a hard integrity check. No mistakes this time.

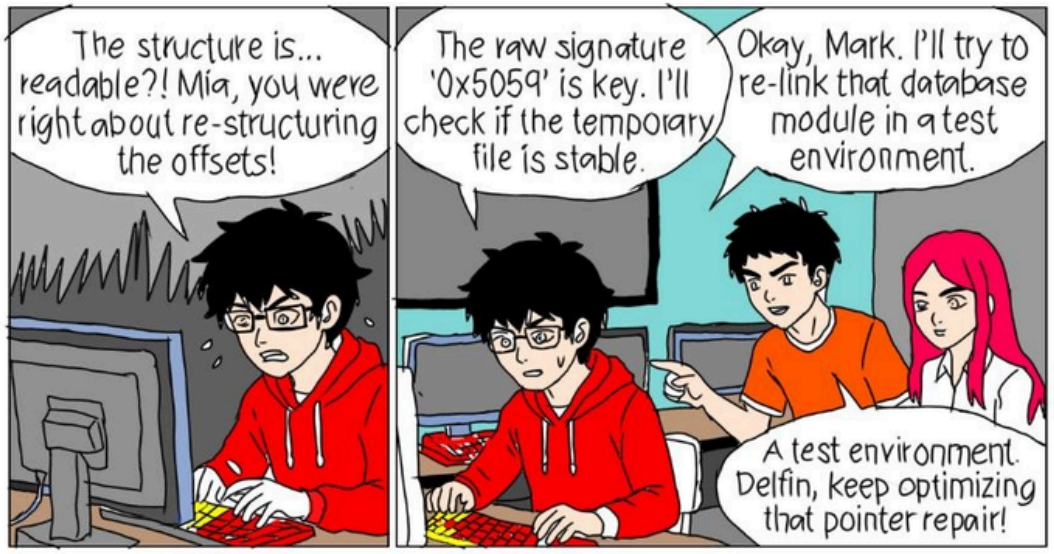

Okay, the direct read is mapping the sectors. How many did we force copy again?
14:49
1024 sectors, Delfin. Focus! If the header is just a few lines offset, we can find it.
The raw signature database search as that '0x5059' *has* to be in there.

14:48
You have to force the copy before the system verification. Use the temporary mount point we setup.
Delfin, that's unstable. Remember Mark's direct read trick?


Delfin, Mark, look! It found the signature! It's not corrupted!
HEX DUMP SEARCH (BETA)
It's exactly as Mark hypothesized. The raw data structure is intact, just mislabeled.
Make sure you include the temporary mount point check.

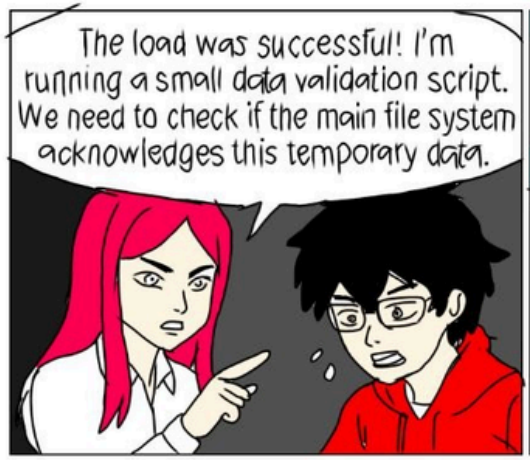

The verification script is looking for the database headers, not just the raw bytes. We must re-structure the raw copy first.
SYSTEM CONSOLE
command> python3 verify_sectors.py
Found structure: [module.bak]
[data_raw] [database_module_fragment]
Matching offsets.. Signature 0x5059 confirmed.


It's exactly as Mark hypothesized. The raw data structure is intact, just mislabeled.
The pointers were just telling the system the wrong starting point!
This is it! Time to write that verify script, Delfn. We've got 18 minutes!
1:44

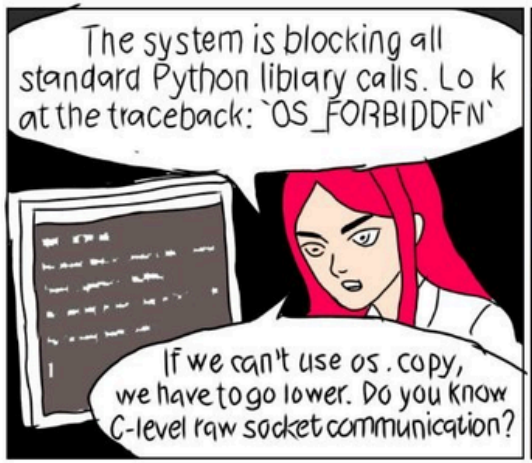

The structure is... readable?! Mia, you were right about re-structuring the offsets!
The raw signature '0x5059' is key. I'll check if the temporary file is stable.
Okay, Mark. I'll try to re-link that database module in a test environment.
A test environment. Delfin, keep optimizing that pointer repair!

The raw signature '0x5059' is holding. If the test environment mounts the database, we can check the integrity.
14:52
I'm Pushing The data now
It has to. The offsets and pointer structure were confirmed.

The system wiped the temporary space, but I brought this physical external drive.
We have to bypass the standard file manager. It's too slow.
Right, but we must make a raw sector copy... not just file copying.

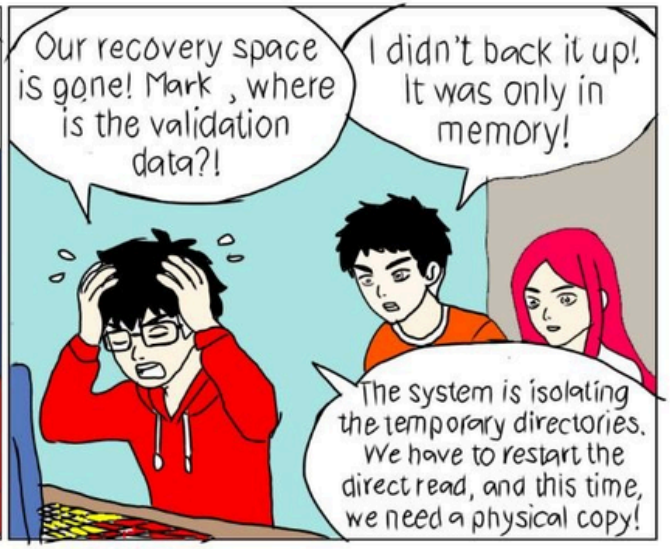

The load was successful! I'm running a small data validation script. We need to check if the main file system acknowledges this temporary data.

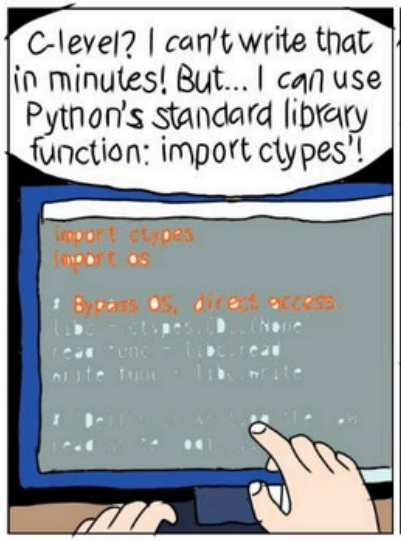
Make sure you include the temporary mount point check.
14:52
Right, checking... `mnt_pt/mark_mount/test`...

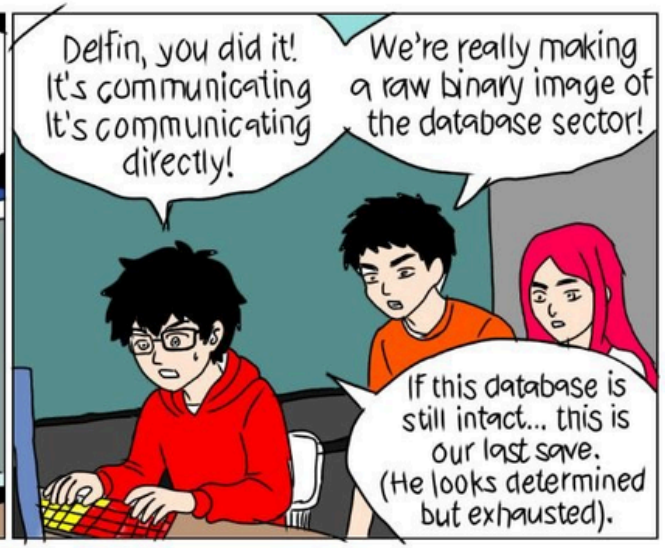

The system is blocking all standard Python library calls. Lo k at the traceback: `OS_FORBIDDFN`
If we can't use os.copy, we have to go lower. Do you know C-level raw socket communication?

It's a kernel-level lock
If I call the raw read and write functions from the C library... we bypass the OS file manager lock!

The system did a hard integrity check! It just wiped our temporary mount point!

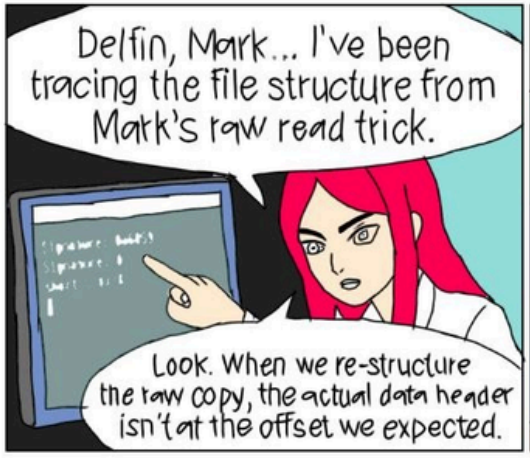

Our recovery space is gone! Mark , where is the validation data?!
I didn't back it up! It was only in memory!
The system is isolating the temporary directories. We have to restart the direct read, and this time, we need a physical copy!


C-level? I can't write that in minutes! But... I can use Python's standard library function: import ctypes'!
import ctypes
import os
# Bypass OS, direct access

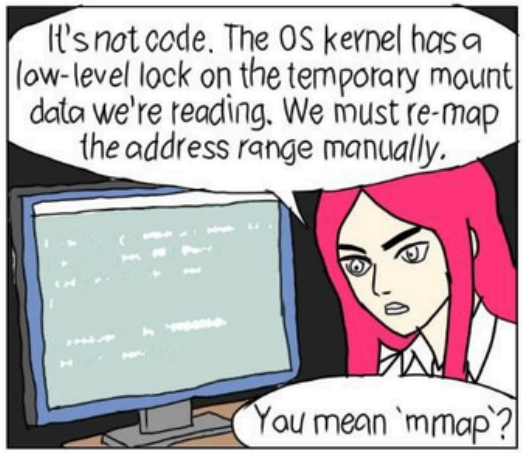

Delfin, you did it! It's communicating It's communicating directly!
We're really making a raw binary image of the database sector!
If this database is still intact... this is our last save. (He looks determined but exhausted).

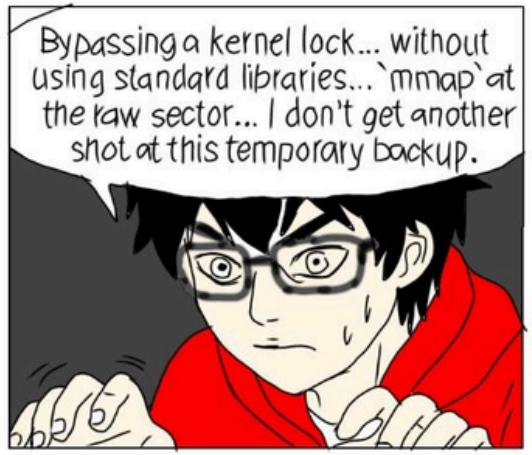
The direct read is successful, Mark. It's pushing the raw bytes.
Great. Now we just need to confirm the database header signature in the stream.

One minute before the hard reset! We must manually verify the kernel offsets.
14:59
Is it the file system pointer check from page 11 agestall again?
I can't re-validate every pointer, I told you! The structure is corrupted!

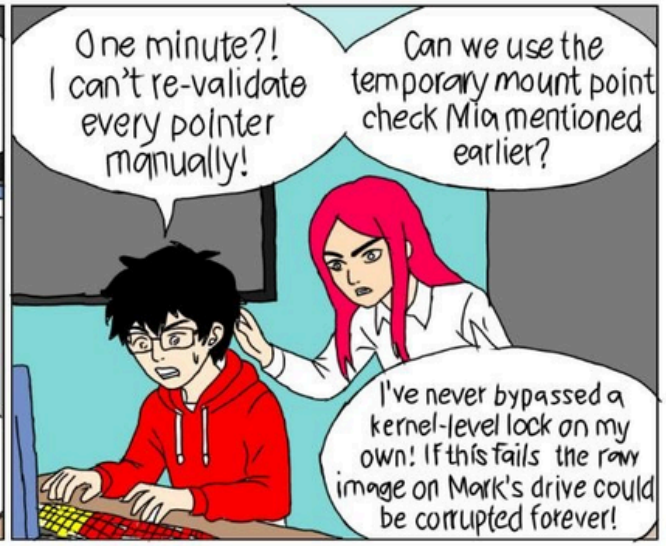

Delfin, Mark... I've been tracing the file structure from Mark's raw read trick.
Look. When we re-structure the raw copy, the actual data header isn't at the offset we expected.

Traced it? What did you find?
It's hidden inside the system's temporary mount point metadata!

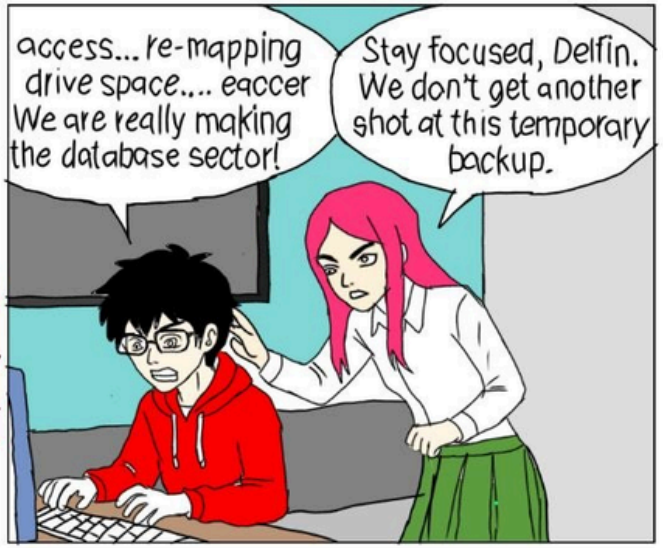

It's not code. The OS kernel has a low-level lock on the temporary mount data we're reading. We must re-map the address range manually.
You mean `mmap`?

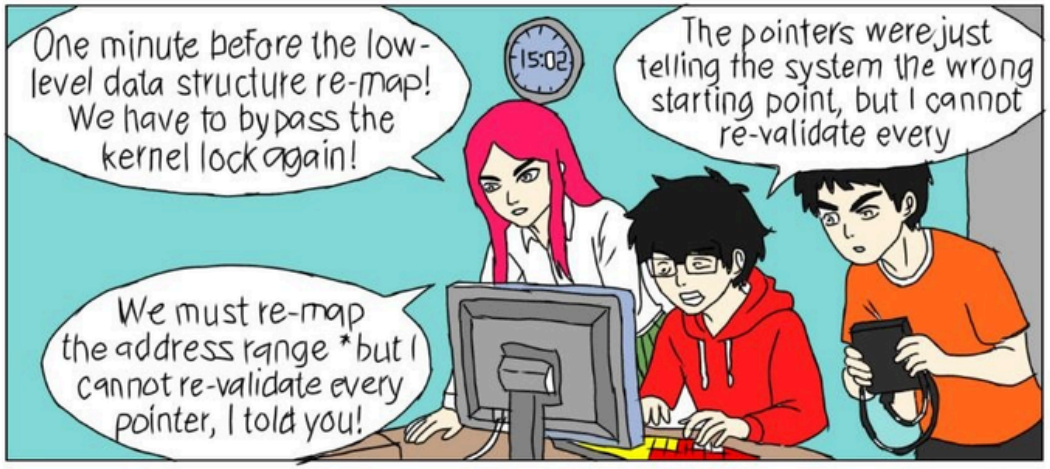

Bypassing a kernel lock... without using standard libraries... `mmap` at the raw sector... I don't get another shot at this temporary backup.

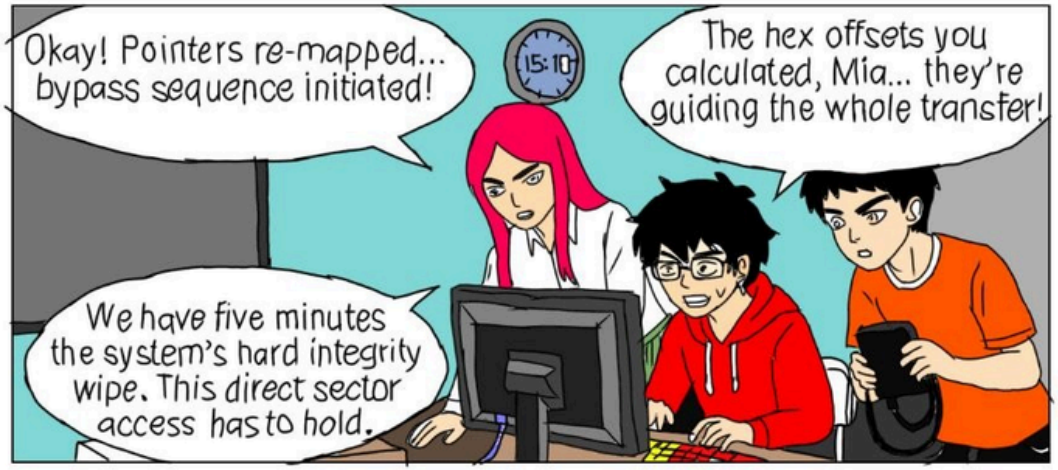
We have one minute and thirty seconds before the system does a hard verification wipe!
SYSTEM INTEGRITY CHECK:
1:30
We have to bypass the to pointer table. Delfin, I need to re-validate the temporary point check right now.

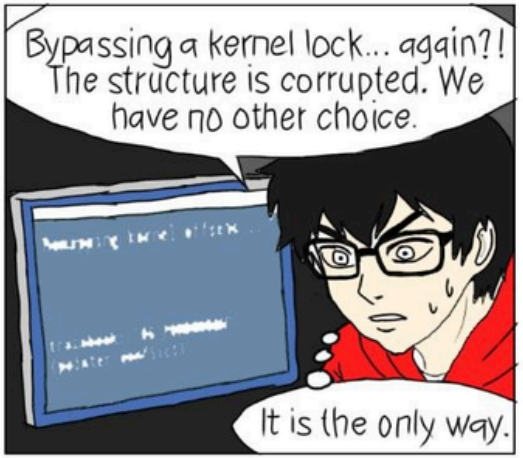

One minute?! I can't re-validate every pointer manually!
Can we use the temporary mount point check Mia mentioned earlier?
I've never bypassed a kernel-level lock on my own! If this fails the raw image on Mark's drive could be corrupted forever!

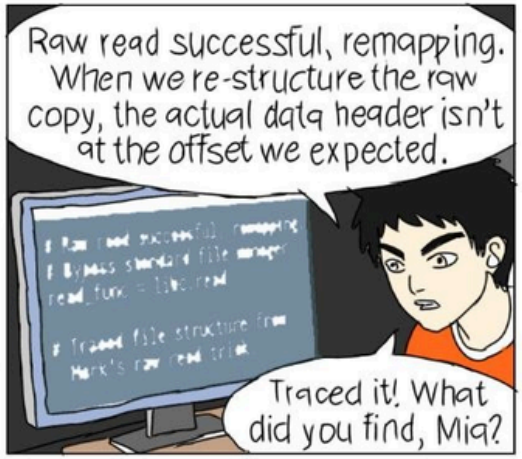
Using `libc.mmap` to force pointers to Mark's physical raw sector by raw sector! a binary image of
We are really making. We don't get another shot at the database backup.


access... re-mapping drive space.... eaccer We are really making the database sector!
Stay focused, Delfin. We don't get another shot at this temporary backup.

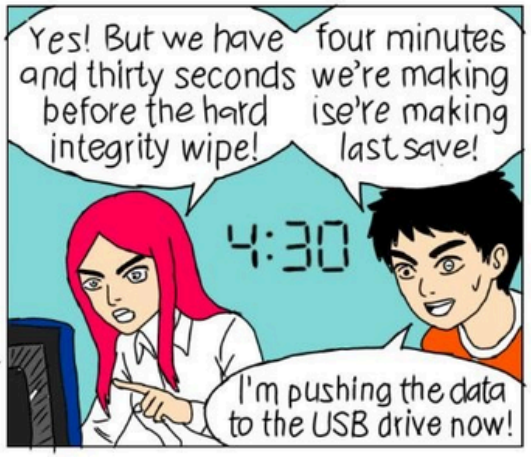

One minute before the low-level data structure re-map! We have to bypass the kernel lock again!
15:02
The pointers were just telling the system the wrong starting point, but I cannot re-validate every
We must re-map the address range * but I cannot re-validate every pointer, I told you!

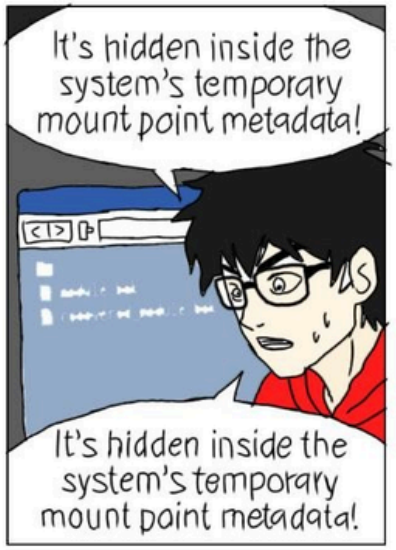

Okay! Pointers re-mapped... bypass sequence initiated!
15:10
The hex offsets you calculated, Mia... they're guiding the whole transfer!
We have five minutes the system's hard integrity wipe. This direct sector access has to hold.

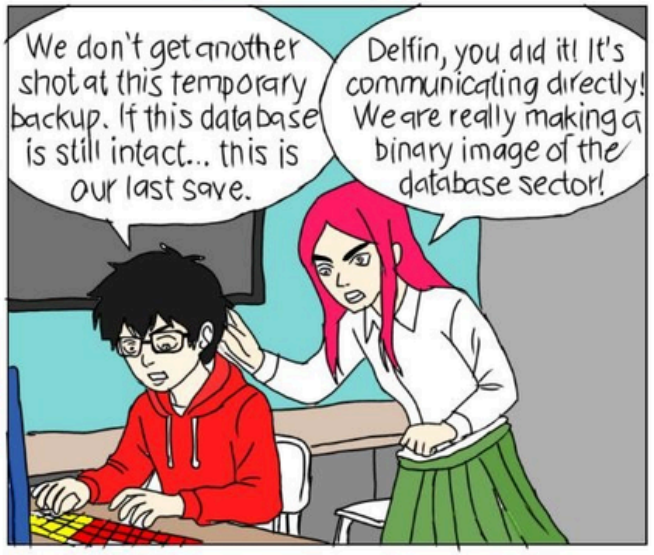

Bypassing a kernel lock... again?! The structure is corrupted. We have no other choice.
It is the only way.

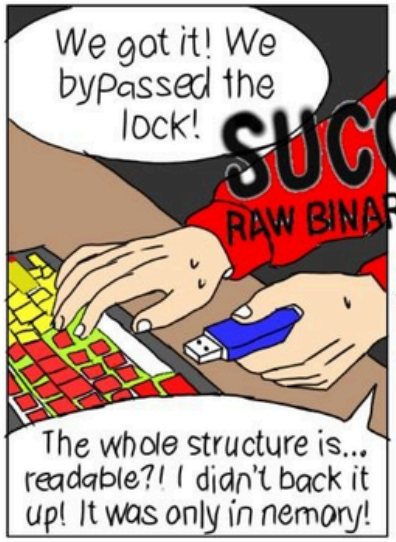

Raw read successful, remapping. When we re-structure the raw copy, the actual data header isn't at the offset we expected.
Traced it! What did you find, Mia?

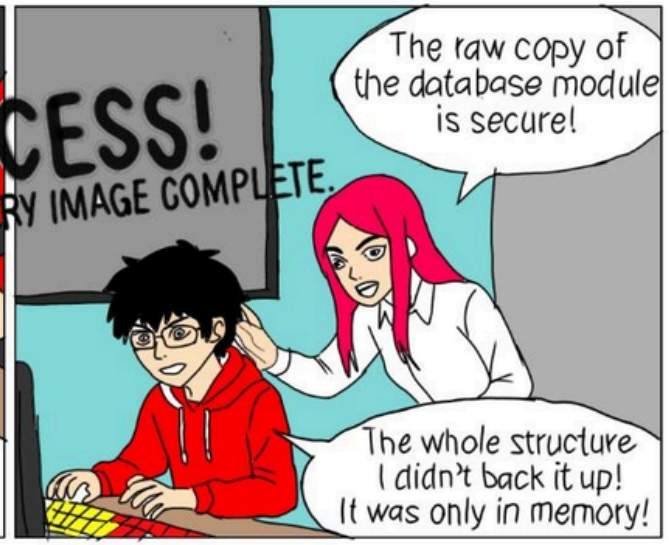
Raw read successful! We're bypassing the OS file manager lock!


Yes! But we have four minutes and thirty seconds we're making before the hard ise're making integrity wipe! last save!
4:30
I'm pushing the data to the USB drive now!

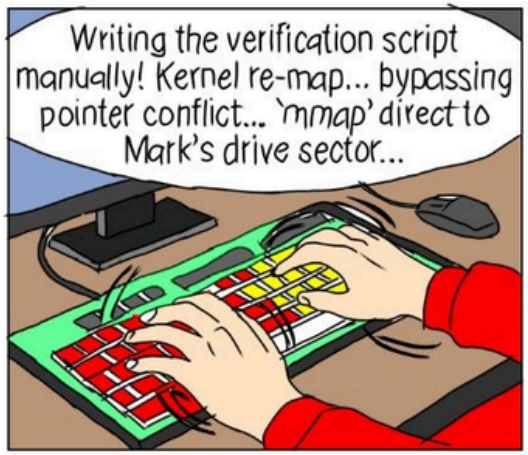

It's hidden inside the system's temporary mount point metadata!
It's hidden inside the system's temporary mount point metadata!

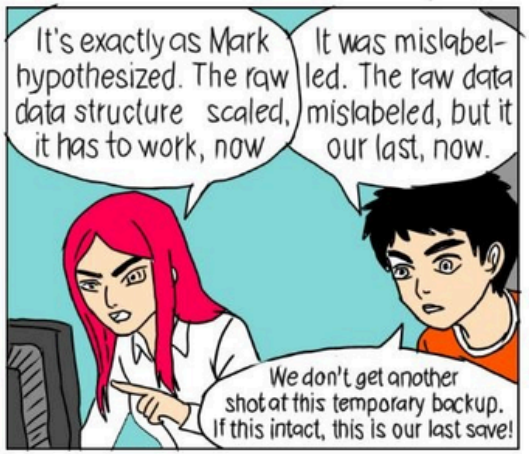

We don't get another shot at this temporary backup. If this database is still intact... this is our last save.
Delfin, you did it! It's communicating directly! We are really making a binary image of the database sector!


We got it! We bypassed the lock!
The whole structure is... readable?! I didn't back it up! It was only in memory!

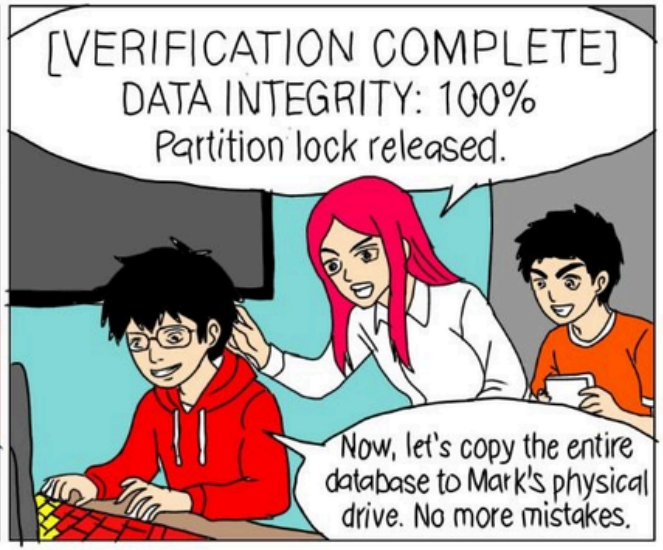

SUCCESS!
RAW BINARY IMAGE COMPLETE.
The raw copy of the database module is secure!
The whole structure I didn't back it up! It was only in memory!

The direct access found the fragment! The signature 0x5059 is correct.
I have to write a script to re-validate every pointer, right now! The OS kernel is still locked.
Almost there, buddy.

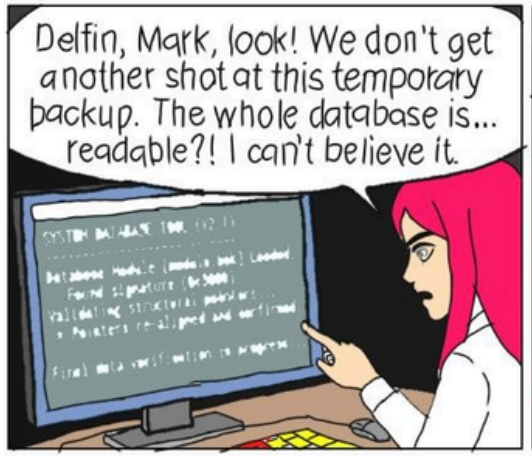

Writing the verification script manually! Kernel re-map... bypassing pointer conflict... 'mmap' direct to Mark's drive sector...

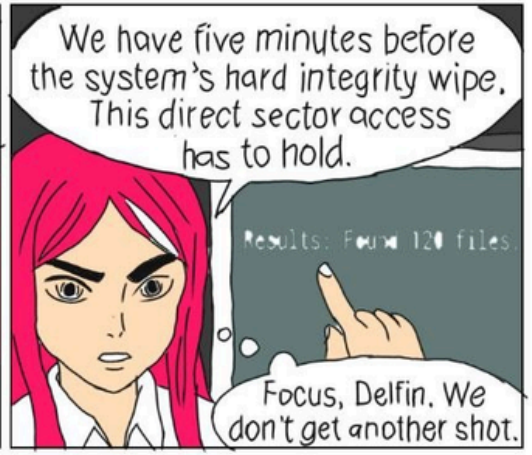

It's exactly as Mark hypothesized. The raw data structure scaled, it has to work, now
It was mislabelled. The raw data mislabeled, but it our last, now.
We don't get another shot at this temporary backup. If this intact, this is our last save!

We got it! We bypassed the lock! The validation data is secure too!
SUCCESS
I didn't back it up, but now the direct read is locked onto the actual file... It was only in memory before.

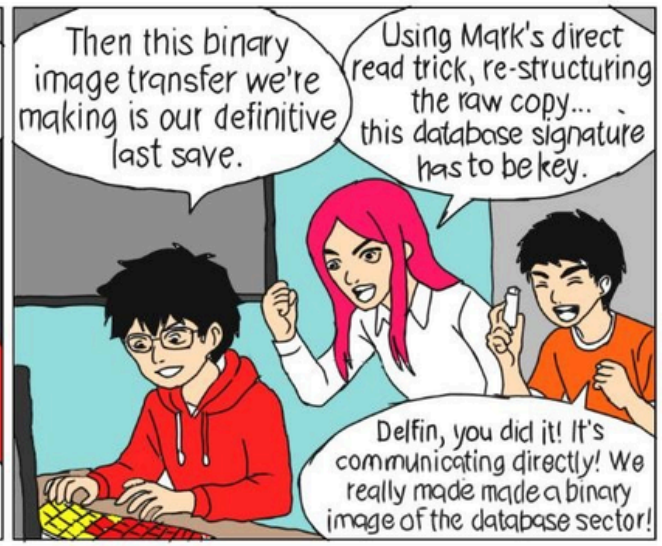

[VERIFICATION COMPLETE]
DATA INTEGRITY: 100%
Partition lock released.
Now, let's copy the entire database to Mark's physical drive. No more mistakes.

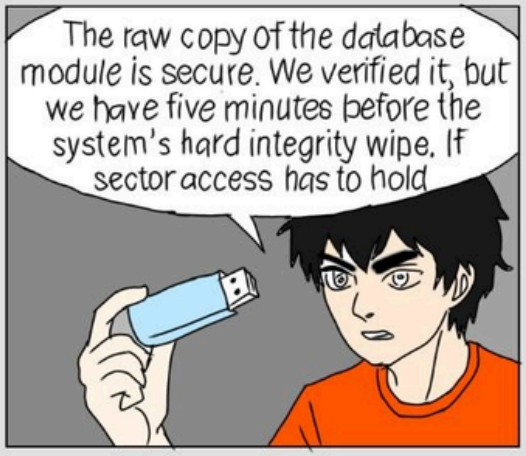
Okay...Integrity check: 100%. We can mount the data on Mark's physical drive and do a final database structure validation.
The direct copy worked! No more raw byte errors. Delfin i's sweat cloand is secure.
The raw copy of the database module is secure.

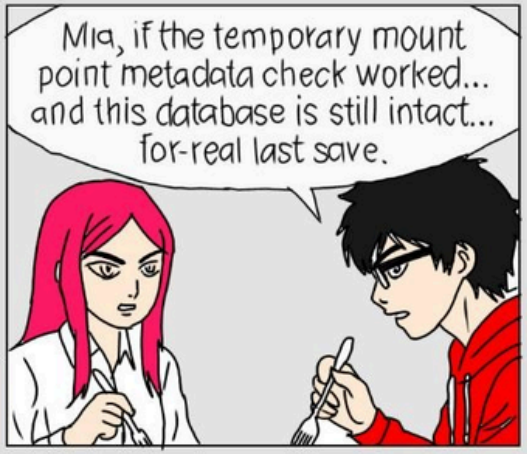

Delfin, Mark, look! We don't get another shot at this temporary backup. The whole database is... readable?! I can't believe it.


We have five minutes before the system's hard integrity wipe. This direct sector access has to hold.
Focus, Delfin. We don't get another shot.

It is the only way. If this database is still intact... this is our actual, for-real last save.
Success! Every sector is copied. Mark's raw copy is secure.


Then this binary image transfer we're making is our definitive last save.
Using Mark's direct read trick, re-structuring the raw copy... this database signature has to be key.
Delfin, you did it! It's communicating directly! We really made made a binary image of the database sector!





Save your work.
Trust your team.
Learn from every mistake.